

Name: A. Abhishek Reddy

Rg no:192424192

DATE:15/7/2026

1. Smart City Air Quality Monitoring: A city has installed 150 air quality sensors across different locations. The sensors record PM2.5, NO2, CO, temperature, and humidity every hour. Decision-makers struggle to identify pollution hotspots because of overlapping values.

A. Python code:

```
import matplotlib.pyplot as plt

PM25 = [40, 60, 80, 100, 120]
NO2 = [20, 30, 40, 50, 60]

plt.scatter(PM25, NO2, c=PM25, cmap="Reds", s=100)

plt.colorbar()

plt.title("Sequential Palette")

plt.show()

Temp = [25, 28, 30, 35, 38]

plt.scatter(PM25, NO2, c=Temp, cmap="coolwarm", s=100)

plt.colorbar()

plt.title("Diverging Palette")

plt.show()

Zone = [1, 2, 3, 4, 5]

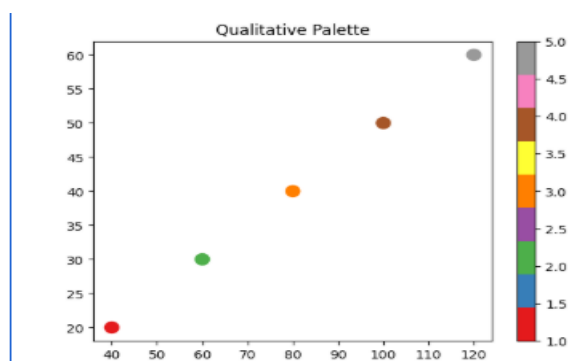
plt.scatter(PM25, NO2, c=Zone, cmap="Set1", s=100)

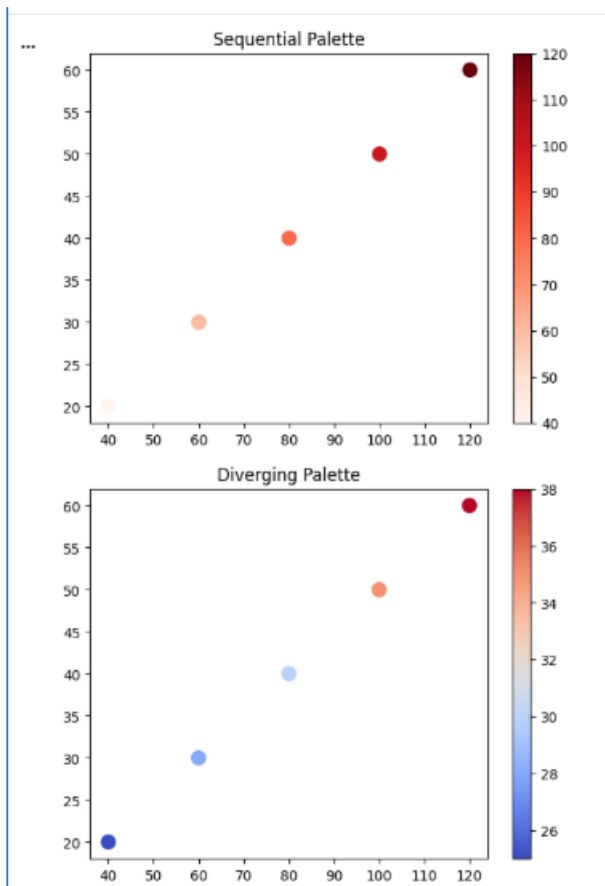
plt.colorbar()

plt.title("Qualitative Palette")

plt.show()
```

OUT PUT:



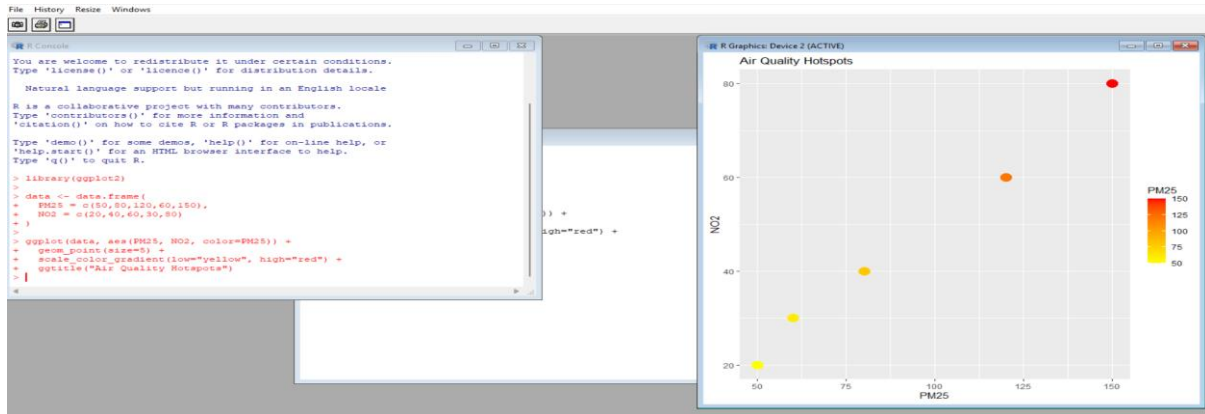


R PROGRAM

```
library(ggplot2)

data <- data.frame(
  PM25 = c(50,80,120,60,150),
  NO2 = c(20,40,60,30,80)
)

ggplot(data, aes(PM25, NO2, color=PM25)) +
  geom_point(size=5) +
  scale_color_gradient(low="yellow", high="red") +
  ggtitle("Air Quality Hotspots")
```



Visualization Design:

- Use a Cartesian scatter plot with PM2.5 on the X-axis and NO2 on the Y-axis.
- Apply different color scales:
 - **Sequential palette** (light to dark red) to represent increasing pollution levels.
 - **Diverging palette** (blue–white–red) to show values below and above the safe pollution limit.
 - **Qualitative palette** to distinguish different city zones or sensor locations.

Comparison:

- **Sequential:** Best for showing pollution intensity from low to high.
- **Diverging:** Best for highlighting safe versus hazardous pollution levels.
- **Qualitative:** Best for comparing different regions but not pollution intensity.

Most Effective Visualization:

The **diverging palette** is the most effective because it clearly separates normal pollution levels from dangerous hotspots, helping decision-makers identify critical areas quickly.

2. Earthquake Risk Analysis: A geological survey contains earthquake magnitude (0.5–9.5), depth (1–700 km), latitude, longitude, and aftershock frequency. Small earthquakes dominate the dataset, making large events difficult to interpret.

A. Python code:

```

import numpy as np

import matplotlib.pyplot as plt

np.random.seed(2)

magnitude=np.random.uniform(0.5,9.5,500)

depth=np.random.randint(1,700,500)

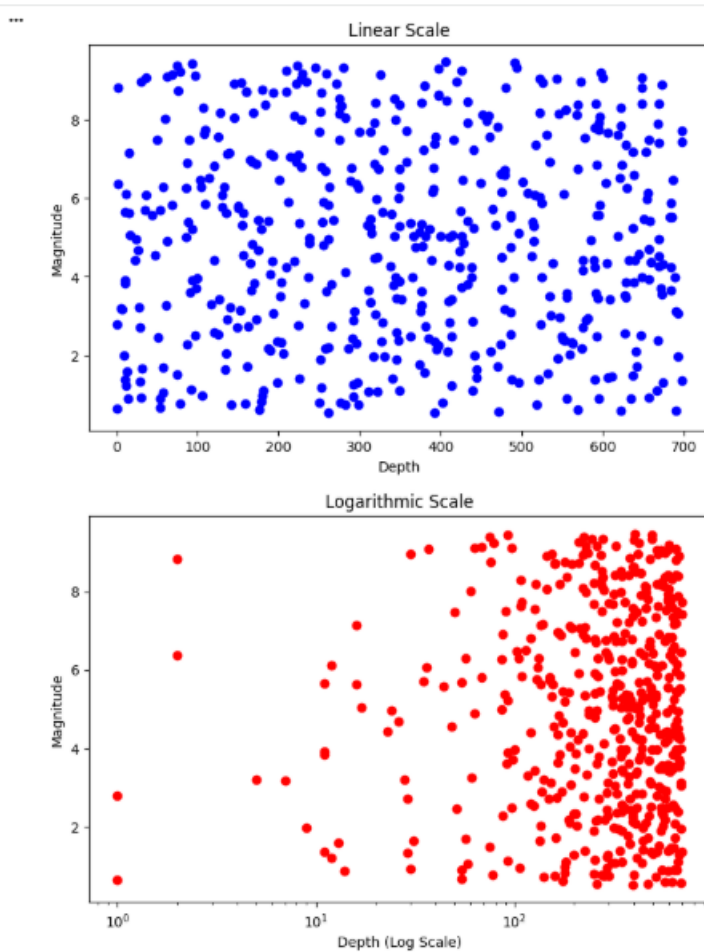
```

```

plt.figure(figsize=(8,5))
plt.scatter(depth,magnitude,color="blue")
plt.xlabel("Depth")
plt.ylabel("Magnitude")
plt.title("Linear Scale")
plt.show()

plt.figure(figsize=(8,5))
plt.scatter(depth,magnitude,color="red")
plt.xscale("log")
plt.xlabel("Depth (Log Scale)")
plt.ylabel("Magnitude")
plt.title("Logarithmic Scale")
plt.show()

```

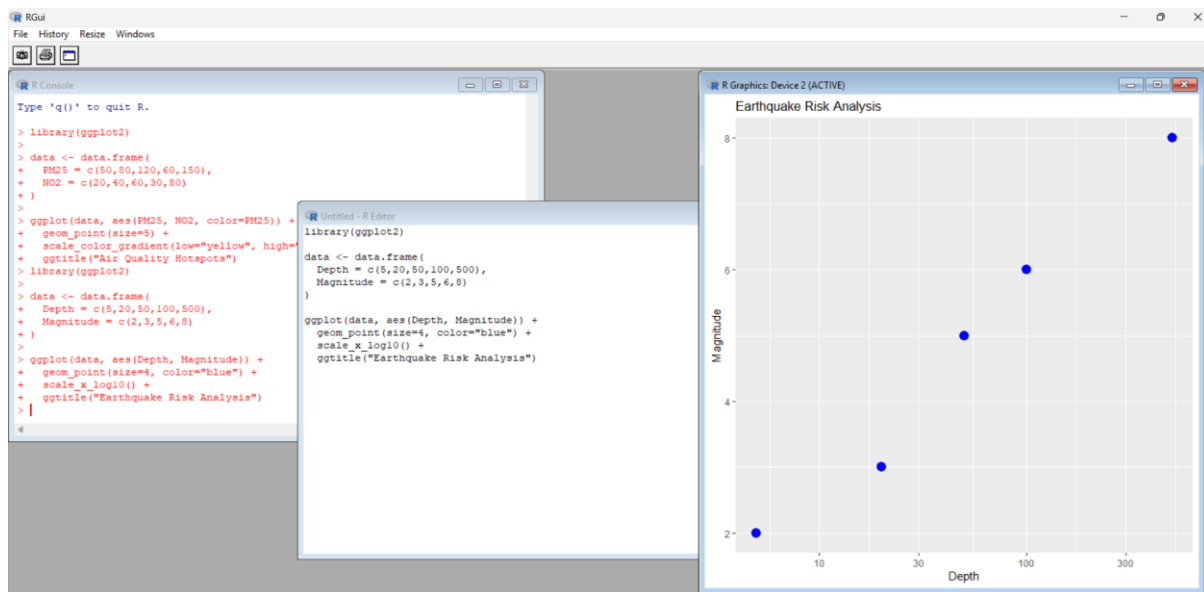


R PROGRAM

```
library(ggplot2)

data <- data.frame(
  Depth = c(5,20,50,100,500),
  Magnitude = c(2,3,5,6,8)
)

ggplot(data, aes(Depth, Magnitude)) +
  geom_point(size=4, color="blue") +
  scale_x_log10() +
  ggtitle("Earthquake Risk Analysis")
```



Visualization Design:

- Create a scatter plot with earthquake **depth** on the X-axis and **magnitude** on the Y-axis.
- Compare:
 - **Linear scale**
 - **Logarithmic (log) scale**

Effect of Logarithmic Scaling:

- Linear scale compresses smaller earthquakes and emphasizes large ones.
- Log scale spreads smaller values, making small and medium earthquakes easier to observe while still displaying larger events clearly.

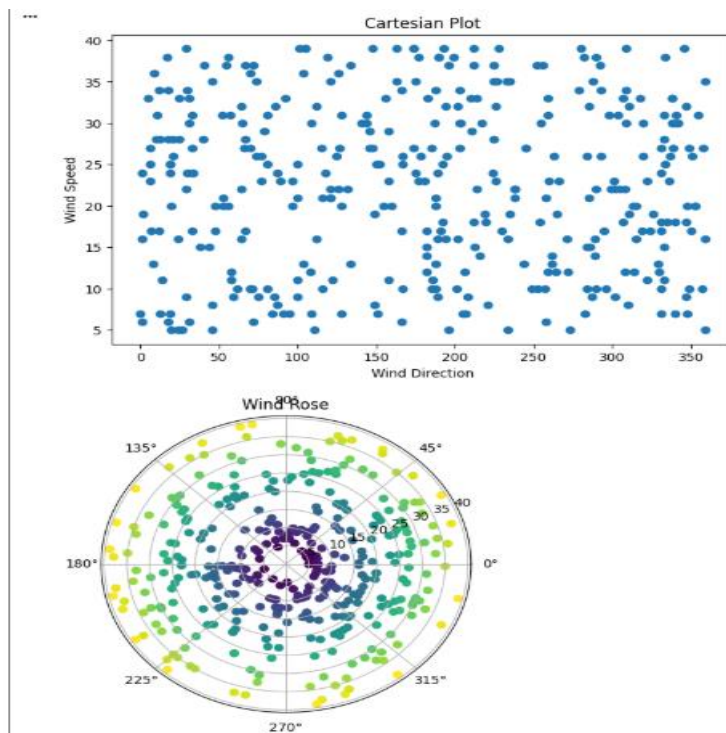
Conclusion:

The logarithmic scale provides a more balanced view of earthquake distribution and makes patterns easier to interpret.

3. Wind Direction Analysis for Airport Operations: An airport collects hourly wind speed and wind direction for one year. Pilots need an intuitive visualization showing prevailing wind directions and seasonal variations.

A. Python code:

```
import numpy as np
import matplotlib.pyplot as plt
direction=np.random.randint(0,360,365)
speed=np.random.randint(5,40,365)
plt.figure(figsize=(8,5))
plt.scatter(direction,speed)
plt.xlabel("Wind Direction")
plt.ylabel("Wind Speed")
plt.title("Cartesian Plot")
plt.show()
ax=plt.subplot(111,projection="polar")
ax.scatter(np.deg2rad(direction),speed,c=speed,cmap="viridis")
plt.title("Wind Rose")
plt.show()
```

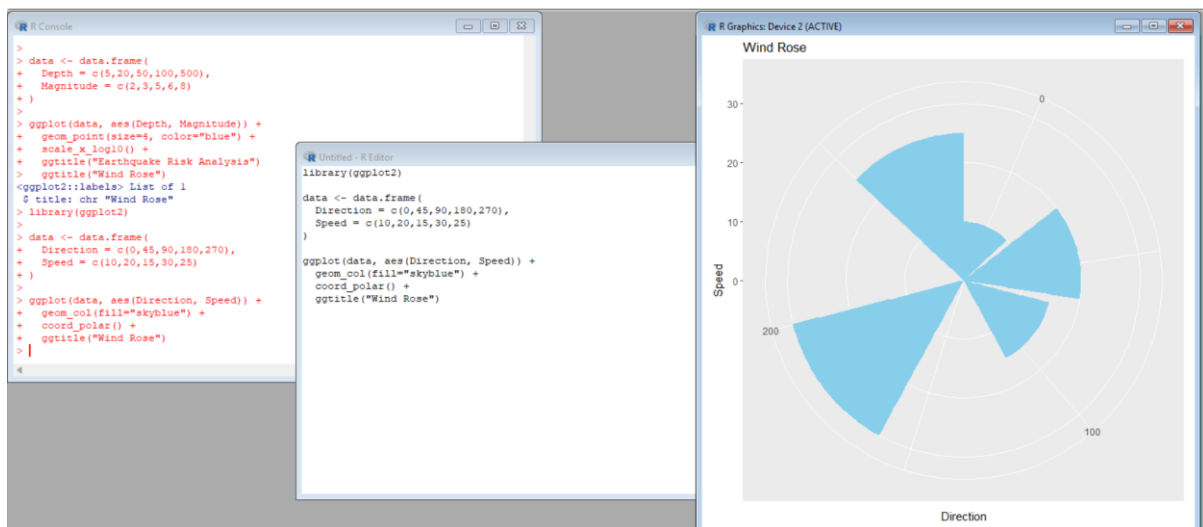


R PROGRAM

```
library(ggplot2)

data <- data.frame(
  Direction = c(0,45,90,180,270),
  Speed = c(10,20,15,30,25)
)

ggplot(data, aes(Direction, Speed)) +
  geom_col(fill="skyblue") +
  coord_polar() +
  ggtitle("Wind Rose")
```



Visualization Design:

- Create a **Cartesian plot** showing wind speed over time.
- Create a **polar (wind rose) chart** showing wind direction and frequency.

Comparison:

- Cartesian plots display changes in wind speed over time.
- Wind rose charts clearly show prevailing wind directions and seasonal patterns.

Best Visualization:

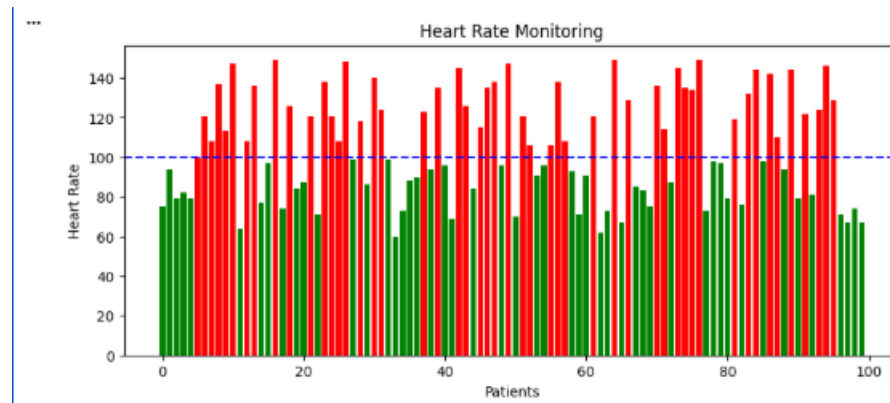
The **wind rose** is more suitable for airport operations because pilots can quickly identify the dominant wind direction for takeoff and landing.

4. Hospital Patient Monitoring Dashboard: ICU patients have heart rate, oxygen saturation, blood pressure, respiration rate, and temperature recorded every minute. Doctors require a

dashboard that instantly highlights abnormal conditions.

A. Python code:

```
import numpy as np
import matplotlib.pyplot as plt
heart=np.random.randint(60,150,100)
colors=["green" if i<100 else "red" for i in heart]
plt.figure(figsize=(10,4))
plt.bar(range(100),heart,color=colors)
plt.axhline(100,color="blue",linestyle="--")
plt.title("Heart Rate Monitoring")
plt.xlabel("Patients")
plt.ylabel("Heart Rate")
plt.show()
```



R PROGRAM

```
library(ggplot2)
data <- data.frame(
  Patient = 1:5,
  HeartRate = c(70,80,120,95,130)
)
ggplot(data, aes(Patient, HeartRate, fill=HeartRate>100)) +
  geom_bar(stat="identity") +
  scale_fill_manual(values=c("green","red")) +
  ggtitle("Heart Rate Dashboard")
```




Visualization Design:

Display patient vital signs using color-coded indicators.

Suggested Thresholds:

- **Green:** Normal values
- **Yellow:** Warning level
- **Red:** Critical condition

Reason for Color Choice:

- Green indicates stability.
- Yellow attracts attention without causing alarm.
- Red immediately highlights emergency conditions requiring medical intervention.

Conclusion:

Using these colors allows doctors to identify abnormal patients instantly without overwhelming the dashboard.

5. Global Climate Change Study: Researchers analyze temperature anomalies across 180 countries from 1980–2025. The objective is to show both positive and negative deviations from historical averages.

A. Python code:

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

data=np.random.uniform(-3,3,(20,10))
```

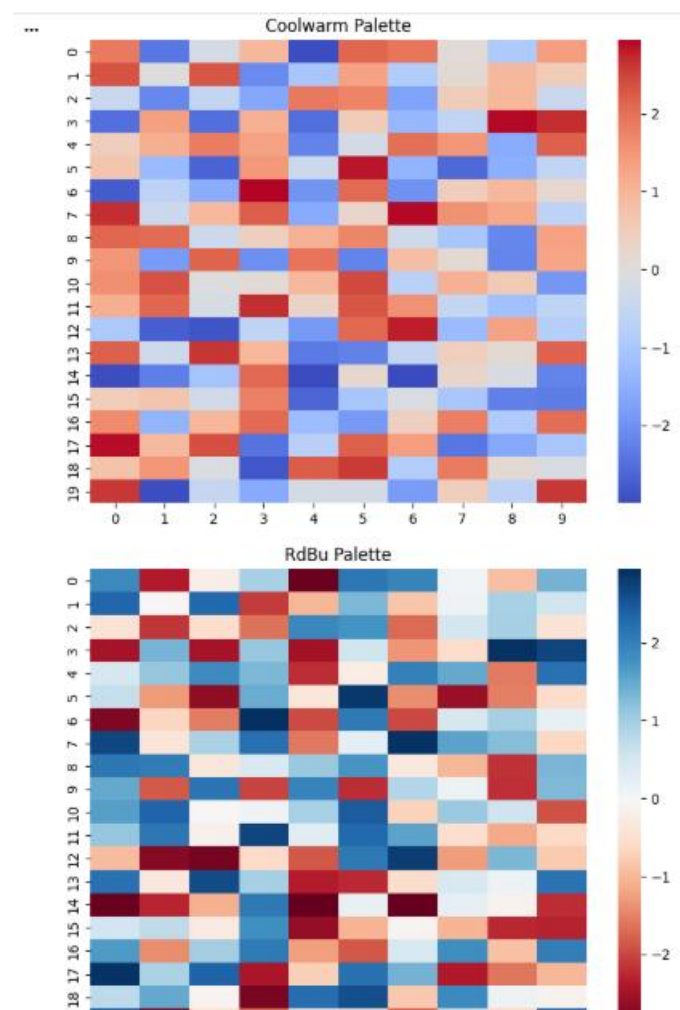
```

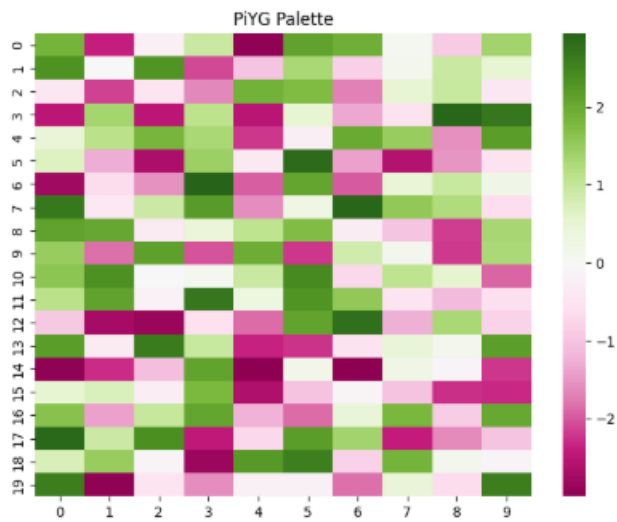
plt.figure(figsize=(8,6))
sns.heatmap(data,cmap="coolwarm")
plt.title("Coolwarm Palette")
plt.show()

plt.figure(figsize=(8,6))
sns.heatmap(data,cmap="RdBu")
plt.title("RdBu Palette")
plt.show()

plt.figure(figsize=(8,6))
sns.heatmap(data,cmap="PiYG")
plt.title("PiYG Palette")
plt.show()

```



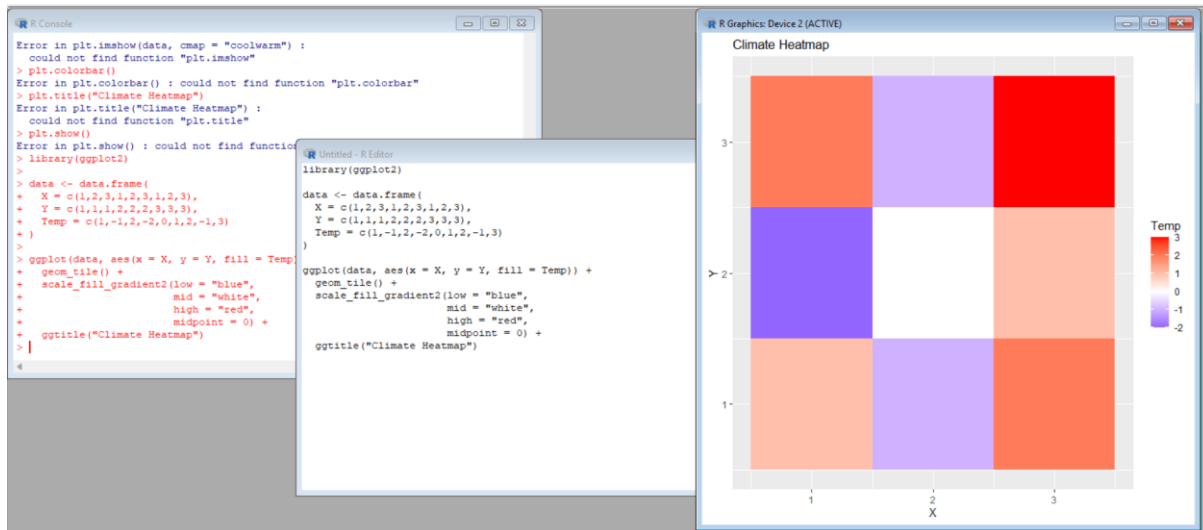


R PROGRAM

```
library(ggplot2)

data <- data.frame(
  X = c(1,2,3,1,2,3,1,2,3),
  Y = c(1,1,1,2,2,2,3,3,3),
  Temp = c(1,-1,2,-2,0,1,2,-1,3)
)

ggplot(data, aes(x = X, y = Y, fill = Temp)) +
  geom_tile() +
  scale_fill_gradient2(low = "blue",
    mid = "white",
    high = "red",
    midpoint = 0) +
  ggtitle("Climate Heatmap")
```



Visualization Design:

Use heatmaps to display temperature anomalies across countries and years.

Diverging Color Palettes:

- Blue–White–Red
- Red–Blue
- Purple–Green

Comparison:

- Blue–White–Red clearly distinguishes cooling and warming.
- Red–Blue provides strong contrast.
- Purple–Green is color-blind friendly.

Most Effective Palette:

Blue–White–Red is the most intuitive because blue naturally represents cooler temperatures while red represents warmer temperatures.

6. E-Commerce Sales Analytics: A retail company sells over 50 product categories across 30 regions. Management wants to distinguish categories while simultaneously representing sales volume.

A. Python code:

```

import pandas as pd
import matplotlib.pyplot as plt

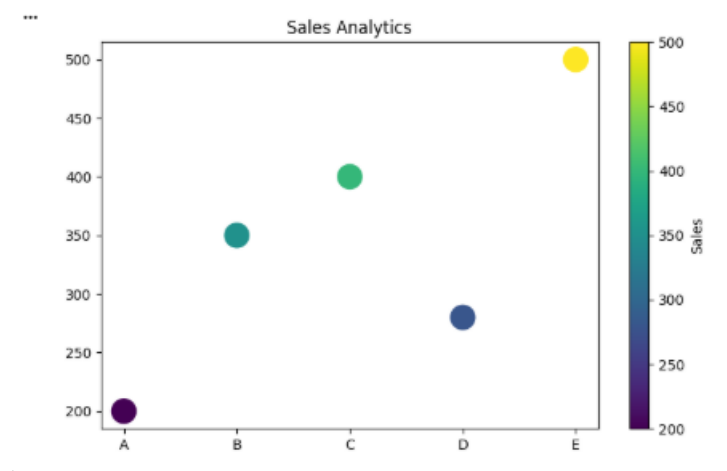
data=pd.DataFrame({
    "Category":["A","B","C","D","E"],

```

```

    "Region":["North","South","East","West","Central"],
    "Sales":[200,350,400,280,500]
})
plt.figure(figsize=(8,5))
plt.scatter(data["Category"],data["Sales"],
            c=data["Sales"],cmap="viridis",s=300)
plt.colorbar(label="Sales")
plt.title("Sales Analytics")
plt.show()

```

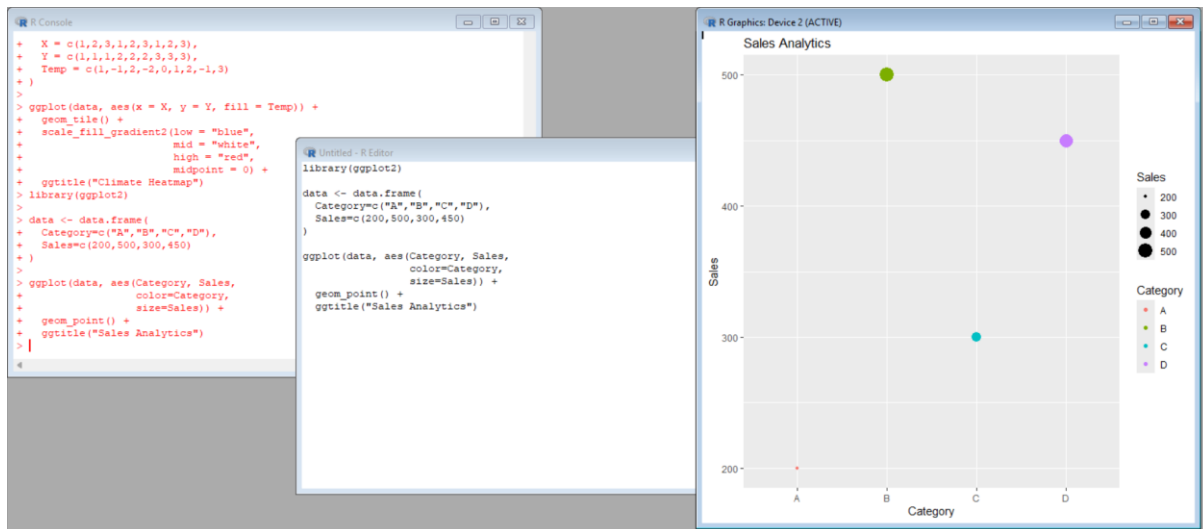


R PROGRAM

```

library(ggplot2)
data <- data.frame(
  Category=c("A","B","C","D"),
  Sales=c(200,500,300,450)
)
ggplot(data, aes(Category, Sales,
                  color=Category,
                  size=Sales)) +
  geom_point() +
  ggtitle("Sales Analytics")

```



Visualization Design:

- Use different colors to represent product categories.
- Use color intensity or point size to represent sales volume.

Challenges:

- Too many colors make categories difficult to distinguish.
- Multiple color meanings may confuse users.
- Similar shades may reduce readability.

Recommendation:

Use **qualitative colors** for product categories and **size or brightness** to represent sales volume for better clarity.

7. Satellite Image Vegetation Analysis: A satellite dataset provides NDVI values ranging from -1 to $+1$. Environmental scientists must identify healthy vegetation, barren land, and water bodies.

A. Python code:

```

import numpy as np
import matplotlib.pyplot as plt

ndvi=np.random.uniform(-1,1,(20,20))

plt.imshow(ndvi,cmap="Greens")

plt.colorbar()

plt.title("Sequential Palette")

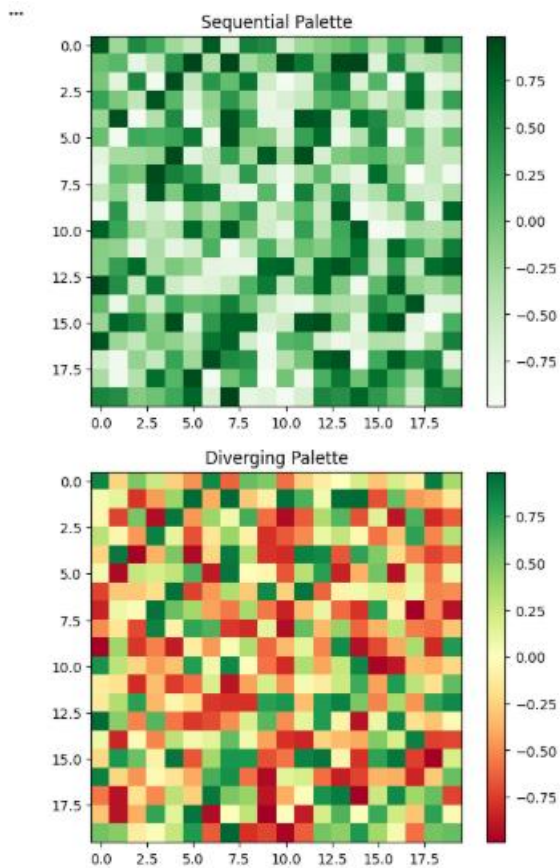
plt.show()

```

```

plt.imshow(ndvi,cmap="RdYlGn")
plt.colorbar()
plt.title("Diverging Palette")
plt.show()

```



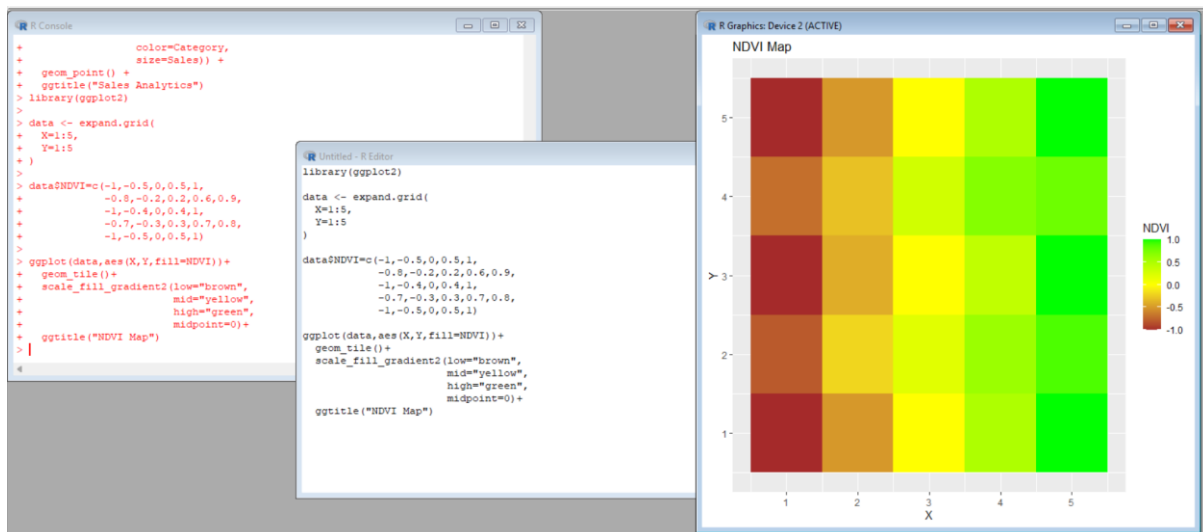
R PROGRAM

```

library(ggplot2)
data <- expand.grid(
  X=1:5,
  Y=1:5
)
data$NDVI=c(-1,-0.5,0,0.5,1,
            -0.8,-0.2,0.2,0.6,0.9,
            -1,-0.4,0,0.4,1,
            -0.7,-0.3,0.3,0.7,0.8,
            -1,-0.5,0,0.5,1)
ggplot(data,aes(X,Y,fill=NDVI))+

```

```
geom_tile()+
scale_fill_gradient2(low="brown",
                     mid="yellow",
                     high="green",
                     midpoint=0)+
ggtitle("NDVI Map")
```



Visualization Design:

Display NDVI values using different color palettes.

Sequential Palette:

- Light green to dark green for vegetation density.

Diverging Palette:

- Brown → Yellow → Green
 - Brown: Barren land
 - Yellow: Moderate vegetation
 - Green: Healthy vegetation

Evaluation:

The diverging palette is more effective because it clearly distinguishes water, barren land, and healthy vegetation while reducing interpretation errors.

8. Cryptocurrency Market Volatility: Daily prices of Bitcoin, Ethereum, Solana, and Cardano span several years. Price values differ by several orders of magnitude, making direct comparison difficult.

A. Python code:

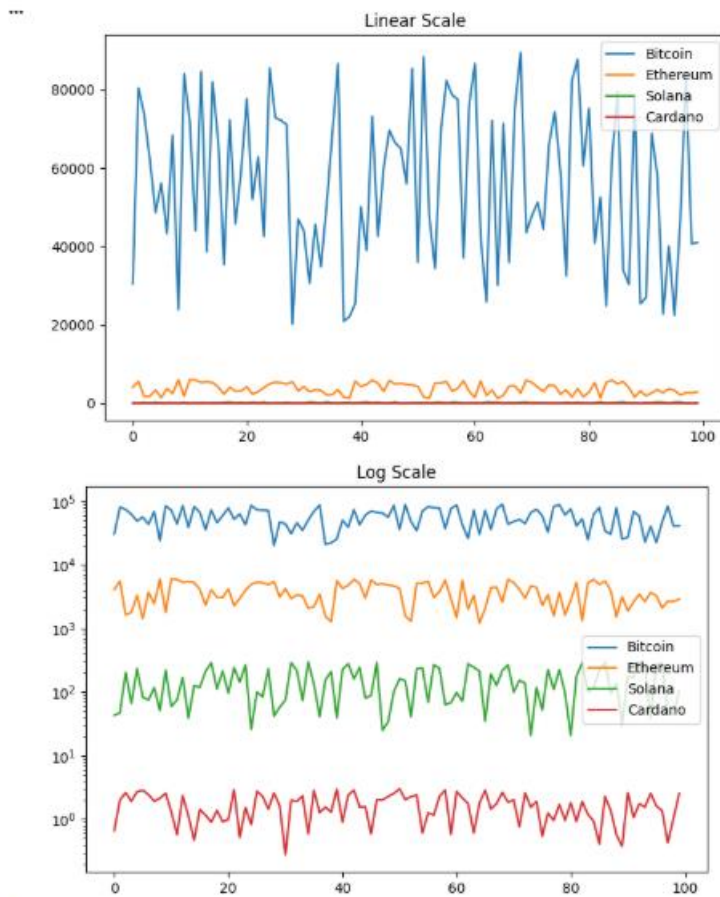
```
import numpy as np
import matplotlib.pyplot as plt

days=np.arange(100)

btc=np.random.randint(20000,90000,100)
eth=np.random.randint(1000,6000,100)
sol=np.random.randint(20,300,100)
ada=np.random.uniform(0.2,3,100)

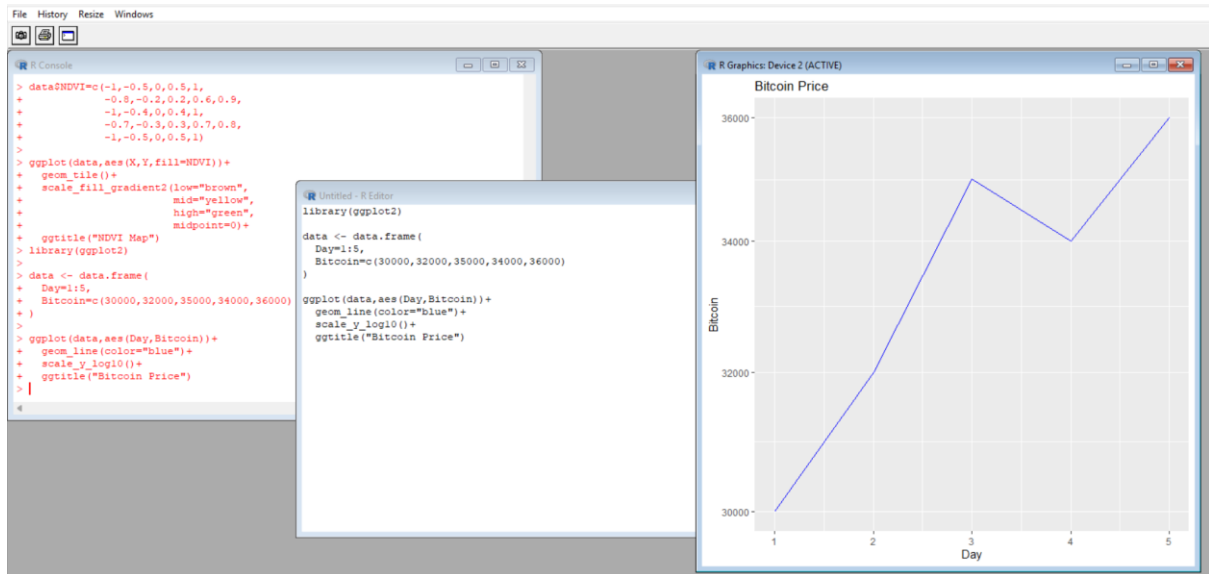
plt.figure(figsize=(8,5))
plt.plot(days,btc,label="Bitcoin")
plt.plot(days,eth,label="Ethereum")
plt.plot(days,sol,label="Solana")
plt.plot(days,ada,label="Cardano")
plt.legend()
plt.title("Linear Scale")
plt.show()

plt.figure(figsize=(8,5))
plt.semilogy(days,btc,label="Bitcoin")
plt.semilogy(days,eth,label="Ethereum")
plt.semilogy(days,sol,label="Solana")
plt.semilogy(days,ada,label="Cardano")
plt.legend()
plt.title("Log Scale")
plt.show()
```



R PROGRAM

```
library(ggplot2)
data <- data.frame(
  Day=1:5,
  Bitcoin=c(30000,32000,35000,34000,36000)
)
ggplot(data,aes(Day,Bitcoin))+
  geom_line(color="blue")+
  scale_y_log10()+
  ggtitle("Bitcoin Price")
```



Visualization Design:

- Plot prices using a standard Cartesian line chart.
- Compare with a logarithmic Y-axis.

Color Usage:

Assign different colors to Bitcoin, Ethereum, Solana, and Cardano.

Benefits of Log Scale:

- Makes assets with vastly different prices easier to compare.
- Highlights percentage changes rather than absolute values.

Limitations:

- Absolute price differences become less obvious.
- May be harder for beginners to interpret.

9. Marathon Runner Performance Analysis: A marathon organizer records runner speed, heart rate, distance covered, elevation, and running direction throughout the race. Coaches want to analyze pacing strategies and directional movement.

A. Python code:

```

import numpy as np

import matplotlib.pyplot as plt

distance=np.linspace(0,42,100)

speed=np.random.uniform(8,15,100)

direction=np.random.randint(0,360,100)

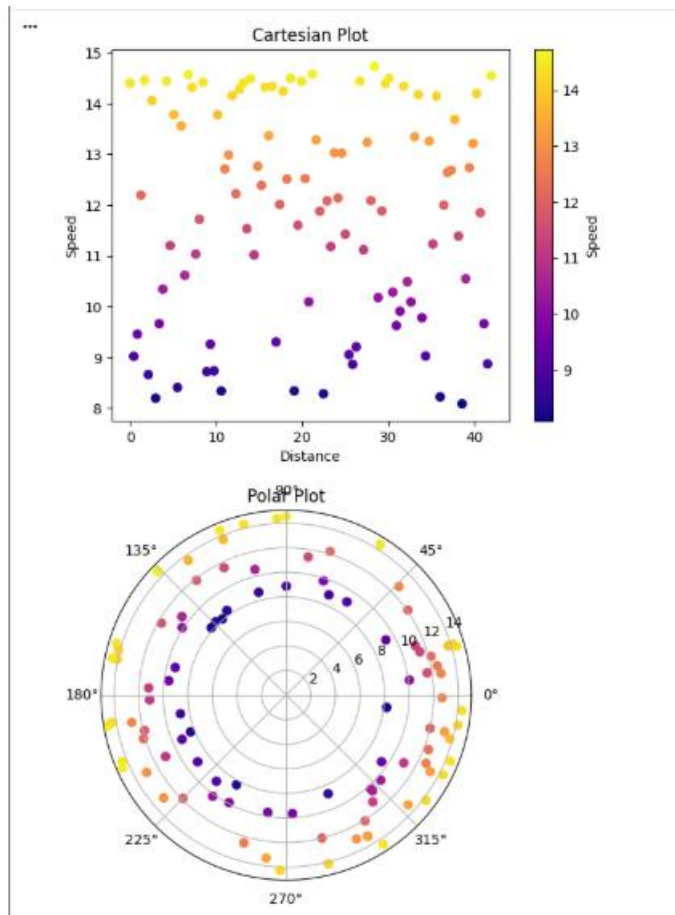
```

```

plt.scatter(distance,speed,c=speed,cmap="plasma")
plt.colorbar(label="Speed")
plt.xlabel("Distance")
plt.ylabel("Speed")
plt.title("Cartesian Plot")
plt.show()

ax=plt.subplot(111,projection="polar")
ax.scatter(np.deg2rad(direction),speed,c=speed,cmap="plasma")
plt.title("Polar Plot")
plt.show()

```



R PROGRAM

```

library(ggplot2)
data <- data.frame(
  Distance=c(5,10,15,20,25),

```

Speed=c(8,9,10,9,8)

)

```
ggplot(data,aes(Distance,Speed,color=Speed))+  
  geom_point(size=5)+  
  scale_color_gradient(low="yellow",high="red")+  
  ggtitle("Runner Performance")
```



Visualization Design:

- Cartesian plot showing speed versus distance.
- Polar plot showing running direction.
- Use color gradients to indicate speed variations.

Comparison:

- Cartesian plot is useful for analyzing pacing.
- Polar plot effectively shows movement direction.

Recommendation:

Using both plots together provides the deepest performance insights by combining pacing analysis with directional movement.

10.COVID-19 Vaccination Campaign Dashboard: Health officials monitor vaccination rates across districts, age groups, and vaccine types. They need visualizations suitable for both policymakers and the general public.

A. Python code:

```
import pandas as pd
```

```

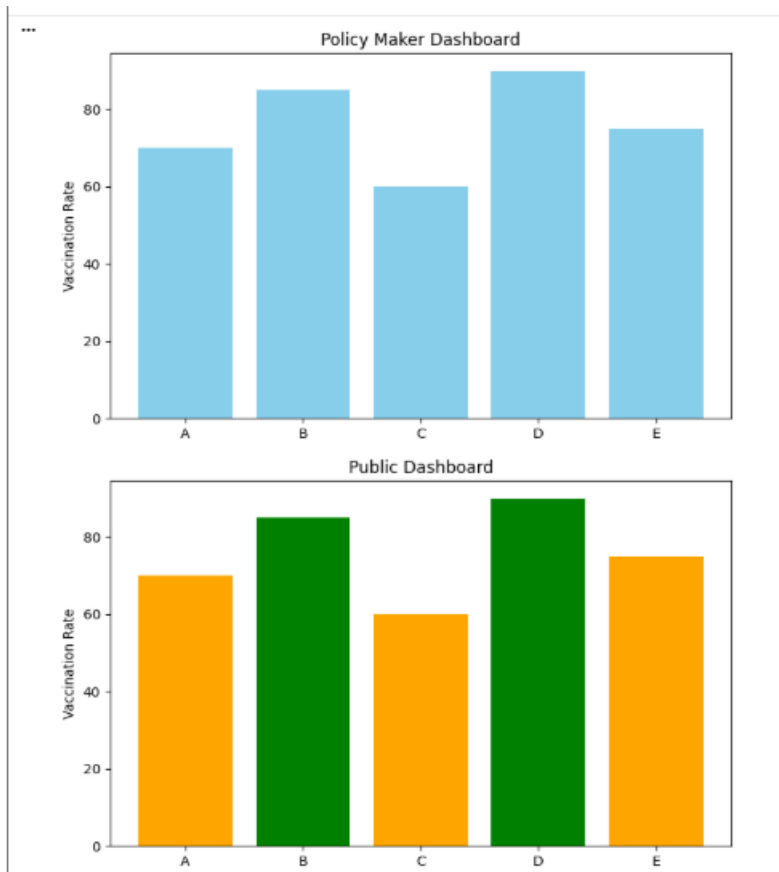
import matplotlib.pyplot as plt

data=pd.DataFrame({
    "District":["A","B","C","D","E"],
    "Rate":[70,85,60,90,75]
})

plt.figure(figsize=(8,5))
plt.bar(data["District"],data["Rate"],color="skyblue")
plt.title("Policy Maker Dashboard")
plt.ylabel("Vaccination Rate")
plt.show()

colors=["green" if i>=80 else "orange" for i in data["Rate"]]
plt.figure(figsize=(8,5))
plt.bar(data["District"],data["Rate"],color=colors)
plt.title("Public Dashboard")
plt.ylabel("Vaccination Rate")
plt.show()

```



R PROGRAM

```
library(ggplot2)

data <- data.frame(

  District=c("A","B","C","D"),

  Rate=c(70,90,60,85)

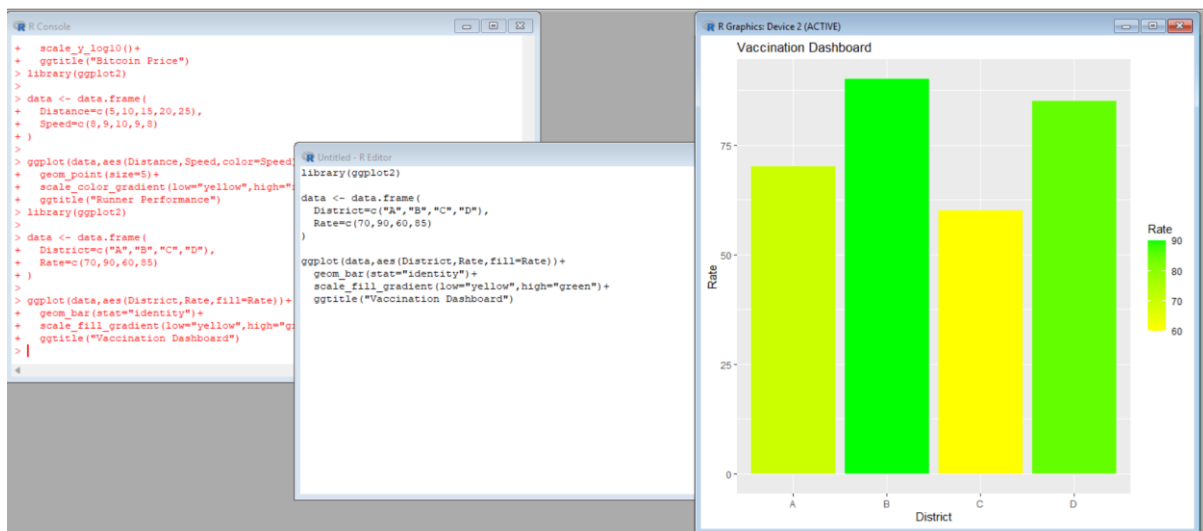
)

ggplot(data,aes(District,Rate,fill=Rate))+

  geom_bar(stat="identity")+

  scale_fill_gradient(low="yellow",high="green")+

  ggtitle("Vaccination Dashboard")
```



Version 1 – Policymakers

- Use detailed charts with sequential color gradients.
- Display district-wise vaccination rates, age groups, and vaccine types.
- Highlight districts with low vaccination coverage.

Version 2 – General Public

- Use simple bar charts and clear color coding.
- Green indicates high vaccination coverage.
- Yellow indicates moderate coverage.
- Red indicates low coverage.

Accessibility Considerations

- Avoid using red and green alone because they are difficult for people with color vision deficiencies.

- Use blue–orange or purple–yellow palettes with labels and legends to improve readability.

Conclusion

The policymaker dashboard should provide detailed analytical information, while the public dashboard should focus on simplicity, clarity, and accessibility for all users.